

一、项目基础信息说明

1.1 项目仓库地址

GitHub 仓库地址：<https://github.com/Nozomi9967/face-detect/tree/main/face-detect-app>

1.2 技术栈

本系统采用的技术栈如下：

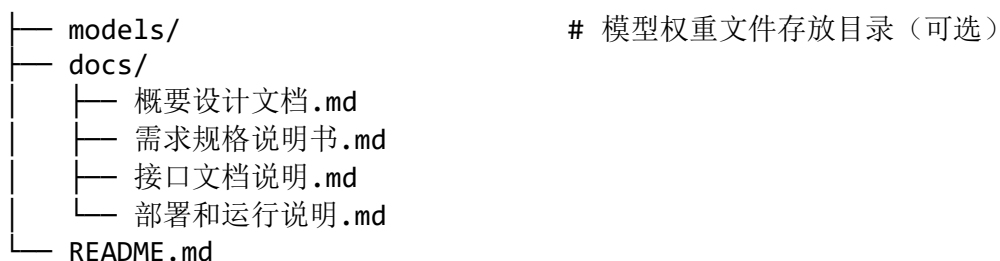
后端框架为 FastAPI + Uvicorn，提供 RESTful API 与异步文件处理能力。人脸检测引擎采用 YOLOv8-face（基于 ultralytics 库）作为主检测器，OpenCV Haar Cascade 作为降级回退检测器。人脸识别引擎采用纯 OpenCV 离线方案，通过灰度缩放、直方图均衡化与 L2 归一化生成特征向量，并以余弦相似度完成跨图人脸匹配（架构设计兼容 InsightFace ArcFace 512 维特征，当前实现为轻量级像素特征方案）。座位分类模块基于人脸框中心坐标的比例规则引擎。前端采用 HTML5 + CSS3 + Vanilla JavaScript 单页应用。图像标注渲染使用 Pillow 库以支持中文标签。

1.3 整体业务目标

本系统面向“多人脸采集素材自动化清洗”场景，核心目标是对车载摄像头拍摄的多角度座舱图片进行自动化人脸检测、身份匹配与座位区域分类。系统支持单张检测与批量清洗两种业务模式：单张模式用于即时检测与结果预览；批量模式用于多张图片的跨图人脸聚类匹配，统一分配人员 ID，并生成 ，辅助数据采集团队高效清洗与标注座舱人脸素材。

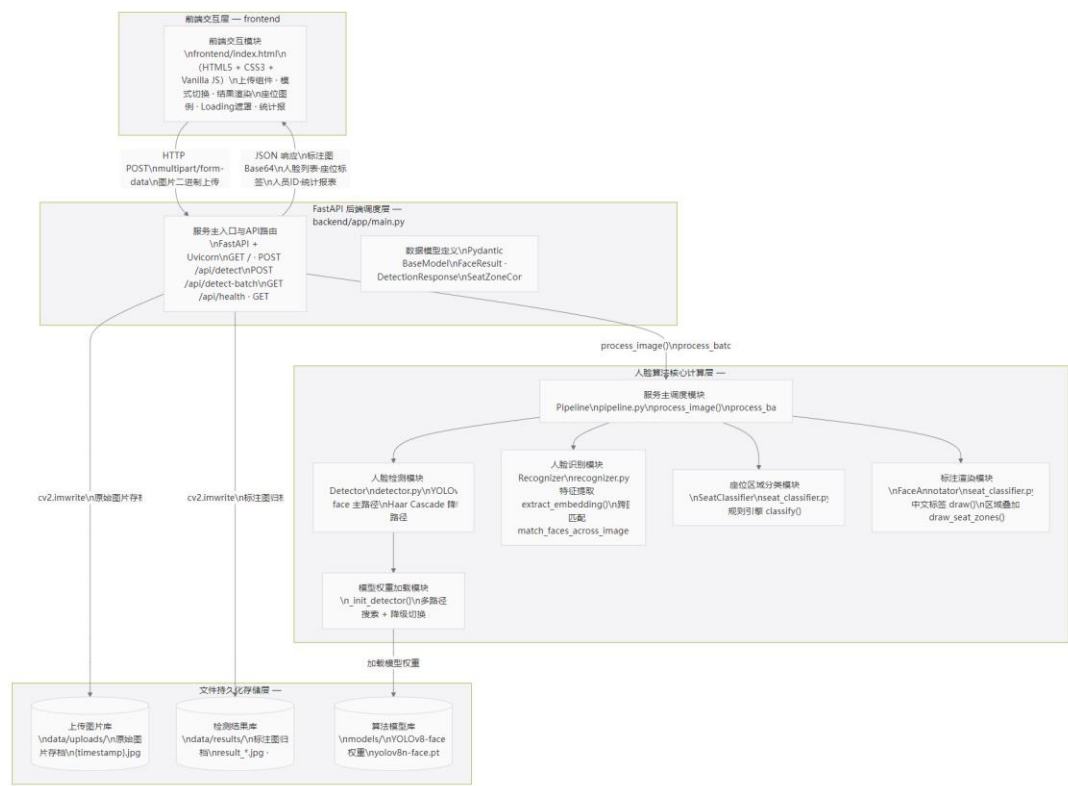
1.4 仓库目录结构

```
face-detect-app/
├── backend/
│   ├── __init__.py
│   ├── requirements.txt
│   └── app/
│       ├── __init__.py
│       ├── main.py                # FastAPI 服务入口 + API 路由定义
│       └── detector.py           # 人脸检测器（YOLOv8-face / Haar Cascade）
├── recognizer.py                 # 人脸识别器（像素特征匹配）
├── seat_classifier.py            # 座位分类器 + 标注渲染器
├── pipeline.py                  # 检测处理管线（调度全流程）
├── frontend/
│   └── index.html               # 前端单页应用（上传+展示）
├── data/
│   ├── uploads/                # 上传原图临时存储（运行时自动创建）
│   └── results/                # 检测标注结果图存储（运行时自动创建）
```



二、系统分层架构图

本系统采用四层分层架构设计，自上而下依次为：前端交互层、FastAPI 后端调度层、人脸算法核心计算层、文件持久化存储层。层与层之间通过明确的接口边界通信，上层依赖下层提供的服务，下层不感知上层存在。以下架构图展示了四层结构中包含的全部功能模块及其层间调用关系。



架构层级说明：前端交互层负责用户操作与结果展示，通过 HTTP 协议与后端通信；FastAPI 后端调度层负责请求路由、图片解码、文件存储与响应封装，是整个系统的调度中枢；人脸算法核心计算层由 Pipeline 统一编排，串联检测、识别、分类、标注四个核心算法模块；文件持久化存储层提供原始图片、检测结果、模型权重三类数据的磁盘读写能力。

三、系统分层功能模块完整拆解

本系统采用四层分层架构设计，自上而下依次为：前端交互层、FastAPI 后端调度层、人脸算法核心计算层、文件持久化存储层。以下逐层拆解各子模块的职责、输入输出、依赖关系与异常处理逻辑。

3.1 前端交互层模块明细

3.1.1 前端交互模块

对应源码文件：`frontend/index.html`

模块类型：基础必选模块

模块核心职责：提供用户可视化操作界面，承载图片上传、模式切换（单张检测 / 批量清洗）、检测结果展示（标注图片渲染、人脸列表、座位分类标签、
等全部前端交互功能。该模块为纯前端单页应用，HTML、CSS、JavaScript 全部内聚于单个 `index.html` 文件中。

输入数据：用户通过点击上传区域或拖拽操作上传的图片文件（JPG/PNG 格式）；用户在“单张检测”与“批量清洗”两个模式标签之间的切换指令。

输出数据：通过 `fetch` API 向后端发送 `multipart/form-data` 格式的 HTTP POST 请求，携带图片二进制数据；接收后端返回的 JSON 格式检测结果并在页面渲染，包括标注后的人脸框图片（Base64 编码）、人脸数量、各人脸的座位分类标签、置信度、人员 ID、处理耗时，以及批量模式下的跨图

依赖上游模块：无（前端为用户操作的起始点）。

依赖下游模块：FastAPI 后端调度层的 `POST /api/detect`（单张）和 `POST /api/detect-batch`（批量）接口。

异常处理逻辑：当 `fetch` 请求抛出网络异常或后端返回非 200 状态码时，前端通过 `try-catch` 捕获错误，以浏览器原生 `alert` 弹窗形式向用户展示错误信息（如“检测失败”或“批量处理失败”）。当图片解析返回 `num_faces === 0` 时，前端以“未检测到人脸”提示卡片代替空结果列表。上传过程中展示全屏半透明 Loading 遮罩与旋转动画，并显示当前处理状态文字（如“正在检测人脸...”、“正在处理 N 张图片...”），处理完成后自动隐藏。

功能子组件说明：前端模块内部包含以下功能子组件——上传区域组件（支持点击选择与拖拽上传，绑定 `dragover`、`dragleave`、`drop` 事件）、模式切换组件（单张/批量标签页，切换时动态调整 `input.multiple` 属性与提示文字）、单张结果渲染组件（`renderSingleResult` 函数，渲染标注图片、人脸列表与置信度条形图）、批量结果渲染组件（`renderBatchResult` 函数，渲染统计汇总表、人员出现次数、

主要座位、所有标注图片卡片）、座位图例组件（展示五个座位区域的颜色映射与中文标签）。

3.2 FastAPI 后端调度层模块明细

3.2.1 服务主入口与 API 路由模块

对应源码文件：backend/app/main.py

模块类型：基础必选模块

模块核心职责：作为整个后端服务的启动入口与请求调度中心。负责创建 FastAPI 应用实例、注册全部 API 路由、初始化全局 Pipeline 引擎实例、配置前端静态文件挂载、处理 HTTP 请求的图片解码与响应编码，以及协调上传文件的磁盘存储与检测结果的持久化保存。

输入数据：前端发送的 HTTP 请求，包括 GET /（主页请求）、GET /api/health（健康检查）、POST /api/detect（单张图片上传，UploadFile 类型）、POST /api/detect-batch（批量图片上传，list[UploadFile] 类型）、GET /api/seat-zones（座位区域配置查询）。

输出数据：GET / 返回前端 index.html 的 HTML 内容；GET /api/health 返回服务状态 JSON；POST /api/detect 返回 DetectionResponse 结构化 JSON，包含 image_id、filename、num_faces、faces（含 bbox、conf、seat、seat_label、person_id）、annotated_image_base64（Base64 编码的标注图）、processing_time_ms；POST /api/detect-batch 返回批量结果 JSON，包含 total_images、total_faces、processing_time_ms、results（每张图片的检测结果数组）。

依赖上游模块：前端交互层（接收前端 HTTP 请求）。

依赖下游模块：服务主调度模块 Pipeline（backend/app/pipeline.py），通过全局实例 pipeline = Pipeline(detector_conf=0.5, similarity_threshold=0.5) 调用 process_image 和 process_batch 方法。

异常处理逻辑：当上传文件无法被 cv2.imdecode 解析为有效图像时，抛出 HTTPException(status_code=400, detail="无法解析图片文件")；当批量上传中全部文件均无法解析时，抛出 HTTPException(status_code=400, detail="未找到有效图片")。

关键业务逻辑：图片接收后先以时间戳命名保存原始文件至 data/uploads/ 目录；调用 Pipeline 处理后将标注图保存至 data/results/ 目录（单张命名格式 result_{timestamp}.jpg，批量命名格式 batch_{timestamp}_{index}.jpg）；标

注图通过 `cv2.imencode` 编码为 JPEG（质量 85%）后经 `base64.b64encode` 编码为 Base64 字符串嵌入 JSON 响应返回前端。

3.2.2 数据模型定义模块

对应源码文件：`backend/app/main.py`（内嵌 Pydantic 模型定义）

模块类型：基础必选模块

模块核心职责：定义 API 接口的请求与响应数据结构模型，确保 FastAPI 自动进行参数校验与序列化。

数据模型清单：`FaceResult` 模型定义单个人脸检测结果结构，包含 `bbox`: `list[float]`（边界框坐标）、`conf`: `float`（置信度）、`seat`: `str`（座位 ID）、`seat_label`: `str`（座位中文标签）、`person_id`: `int`（统一人员 ID）。`DetectionResponse` 模型定义单张检测完整响应结构，包含 `image_id`、`filename`、`num_faces`、`faces`: `list[FaceResult]`、`annotated_image_base64`、`processing_time_ms`。`SeatZoneConfig` 模型定义座位区域配置结构，用于支持前端动态调整座位分区参数。

3.3 人脸算法核心计算层模块明细

3.3.1 服务主调度模块（Pipeline）

对应源码文件：`backend/app/pipeline.py`

模块类型：基础必选模块

模块核心职责：作为核心算法层的统一调度入口，编排完整的人脸检测处理管线。在初始化阶段创建 `FaceDetector`、`FaceRecognizer`、`SeatClassifier`、`FaceAnnotator` 四个核心组件实例；在运行时按序调度“人脸检测 → 特征提取 → 跨图匹配 → 座位分类 → 可视化标注”五个处理步骤，将各子模块串联为完整的端到端处理流程。

输入数据：`process_image` 方法接收 `image`: `np.ndarray`（OpenCV BGR 格式图像矩阵）与 `image_id`: `int`（图片编号，默认为 0）；`process_batch` 方法接收 `images`: `list[np.ndarray]`（多张图像의列表）。

输出数据：`process_image` 返回字典结构包含 `image_id`、`num_faces`、`faces`（列表，每项含 `bbox`、`conf`、`seat`、`seat_label`、`person_id`）、`annotated_image`（标注后的 numpy 图像数组）。`process_batch` 返回元组 (`results_list`, `annotated_images`)，其中 `results_list` 为每张图片的检测结果字典列表，`annotated_images` 为标注图像列表。

依赖上游模块：FastAPI 后端调度层的 `main.py`（由全局 Pipeline 实例被 API 路由调用）。

依赖下游模块：人脸检测模块 `detector`（`backend/app/detector.py`）、人脸识别模块 `recognizer`（`backend/app/recognizer.py`）、座位分类模块 `seat_classifier`（`backend/app/seat_classifier.py`）。

异常处理逻辑：Pipeline 本身不额外捕获异常，各子模块的异常由各模块内部处理。当 `FaceDetector` 初始化失败（如 Haar Cascade 模型无法加载）时，在构造阶段即抛出 `RuntimeError`。当某张图像的人脸特征提取失败（返回 `None`），`match_faces_across_images` 会为该人脸直接分配新的 `person_id`，保证流程不中断。

单张处理流程详述：步骤一，调用 `self.detector.detect(image)` 获取原始人脸检测框列表；步骤二，遍历每张人脸调用 `self.recognizer.extract_embedding(image, face["bbox"])` 提取特征向量，并标记 `image_id`；步骤三，调用 `self.recognizer.match_faces_across_images` 执行跨图匹配分配统一人员 ID；步骤四，调用 `self.classifier.classify(face["bbox"], w, h)` 对每张人脸进行座位区域分类；步骤五，调用 `self.annotator.draw(image, matched_faces)` 在原图上绘制标注框与中文标签。

批量处理流程详述：首先遍历所有图像逐一执行人脸检测与特征提取，将所有图像的人脸数据汇总至 `all_faces` 列表；然后调用 `match_faces_across_images` 对全部人脸做跨图聚类匹配（这是批量模式的核心优势，跨图匹配可更准确地识别同一人）；最后按 `image_id` 重新分组，为每张图的人脸执行座位分类与标注渲染。

3.3.2 人脸检测模块 (*Detector*)

对应源码文件：`backend/app/detector.py`

模块类型：基础必选模块

模块核心职责：实现图像中的人脸定位检测，输出每个人脸区域的边界框坐标与置信度分数。支持双引擎架构：优先使用 YOLOv8-face 深度学习模型进行高精度检测；当 YOLOv8 模型文件不可用或 `ultralytics` 库未安装时，自动降级至 OpenCV Haar Cascade 级联分类器。

输入数据：`image`: `np.ndarray`（OpenCV BGR 格式图像矩阵），以及可选的 `model_path`: `str`（YOLOv8 模型文件路径）和 `conf_threshold`: `float`（检测置信度阈值，默认 0.5）。

输出数据：`list[dict]` 格式的人脸检测结果列表，每个字典包含 `bbox`: `[x1, y1, x2, y2]`（像素坐标，左上角与右下角）和 `conf`: `float`（检测置信度），结果按 `bbox` 的 `x1` 坐标从左到右排序。

依赖上游模块：Pipeline 模块（由 `pipeline.py` 中的 `self.detector.detect(image)` 调用）。

依赖下游模块：算法模型库 `models/`（加载 YOLOv8 模型权重文件 `yolov8n-face.pt`，或从 `~/.ultralytics/weights/` 缓存目录加载）。

双引擎初始化逻辑（`_init_detector` 方法）：第一优先级，尝试 `from ultralytics import YOLO` 导入 YOLOv8 库，若成功则依次检查用户指定路径、当前目录、`~/.ultralytics/weights/` 缓存目录是否存在模型文件，找到则加载并设置 `self.backend = "yolo"`；第二优先级（降级路径），若 `import` 失败（`ImportError`）或模型文件不存在，则调用 `_init_haar()` 初始化 Haar Cascade 检测器，设置 `self.backend = "haar"`。

Haar Cascade 初始化逻辑（`_init_haar` 方法）：从 `cv2.data.harcascades` 路径加载 `haarcascade_frontalface_default.xml` 级联分类器文件，若加载失败（`cascade.empty()` 返回 `True`）则抛出 `RuntimeError`。

YOLOv8 检测逻辑（`_detect_yolo` 方法）：调用 `self.yolo_model.predict(source=image, conf=self.conf_threshold, verbose=False, device="cpu")` 执行推理，遍历检测结果的 `boxes` 提取 `xyxy` 坐标与置信度，按 `x1` 坐标排序后返回。

Haar Cascade 检测逻辑（`_detect_haar` 方法）：先将图像转为灰度，再调用 `detectMultiScale` 执行多尺度检测（`scaleFactor=1.1`、`minNeighbors=5`、`minSize=(60,60)`），由于 Haar 不返回原始置信度，采用检测面积占全图面积的比例映射为伪置信度（公式： $\min(0.5 + \text{area_ratio} * 10, 0.99)$ ）。

3.3.3 人脸识别模块 (Recognizer)

对应源码文件：`backend/app/recognizer.py`

模块类型：基础必选模块

模块核心职责：实现人脸特征的提取与跨图像身份匹配。采用纯 OpenCV 离线方案，通过灰度缩放、直方图均衡化生成归一化特征向量，以余弦相似度作为匹配度量，在多图场景下为同一人员分配统一的人员 ID。架构设计兼容 InsightFace ArcFace 512 维深度特征提取器（作为高级方案），当前实现为轻量级像素统计特征方案（作为降级/轻量方案）。

输入数据：`extract_embedding` 方法接收 `image: np.ndarray`（完整图像）与 `bbox: list[float]`（人脸边界框坐标）；`match_faces_across_images` 方法接收 `all_faces: list[dict]`（含 `embedding` 与 `image_id` 的人脸列表）与 `similarity_threshold: float`（匹配阈值，默认 0.5）。

输出数据：`extract_embedding` 返回 `Optional[np.ndarray]`（256 维 L2 归一化特征向量，提取失败返回 `None`）；`match_faces_across_images` 返回带有 `person_id` 字段的增强人脸列表。

依赖上游模块：Pipeline 模块（由 `pipeline.py` 调用特征提取与跨图匹配方法）。

依赖下游模块：无直接文件依赖（纯 OpenCV 实现，不依赖外部模型文件）。

特征提取逻辑（`extract_embedding` 方法）：根据 `bbox` 坐标裁剪人脸 ROI 区域（含边界安全检查），缩放至 16x16 像素，转为灰度图，执行直方图均衡化（`cv2.equalizeHist`）以归一化光照条件，展平为 256 维浮点向量，最后进行 L2 归一化。当裁剪区域无效（坐标越界或面积为零）或归一化范数接近零时返回 `None`。

跨图匹配逻辑（`match_faces_across_images` 方法）：按 `image_id` 对人脸进行分组，维护一个 `person_db` 字典（键为 `person_id`，值为该人员所有已知特征向量列表）。按图片顺序遍历每张人脸，计算其特征向量与 `person_db` 中所有已知向量的余弦相似度，选取相似度最高且超过阈值（默认 0.5）的匹配结果，将人脸归入已有人员 ID；若无匹配则分配新的递增 `person_id`。当某人脸的 `embedding` 为 `None` 时，直接分配新 ID。

余弦相似度计算（`cosine_similarity` 静态方法）：对两个特征向量计算点积（因输入已 L2 归一化，点积等价于余弦相似度），任一输入为 `None` 时返回 0.0。

架构扩展说明：根据项目 README 的设计规划，系统架构预留了 InsightFace ArcFace（512 维深度特征向量）的接入点，当部署环境安装 `insightface` 库并下载对应模型文件后，可将 `extract_embedding` 替换为 ArcFace 深度特征提取，获得更高精度的跨图匹配效果。当前实现（像素统计特征）作为轻量级降级方案，无需下载任何外部模型文件即可运行。

3.3.4 座位区域分类模块（*SeatClassifier*）

对应源码文件：`backend/app/seat_classifier.py`（`SeatClassifier` 类）

模块类型：基础必选模块

模块核心职责：基于人脸检测框中心坐标在图像中的归一化位置，通过预定义的规则引擎将人脸分类到五个座舱区域之一。

输入数据：`bbox`: `list[float]`（人脸边界框坐标 `[x1, y1, x2, y2]`）、`img_w`: `int`（图像宽度）、`img_h`: `int`（图像高度）。

输出数据：`str` 类型的座位区域 ID，取值为以下五个枚举值之一：`master_driver`（主驾）、`co_driver`（副驾）、`posterior_left`（后排左）、`posterior_right`（后排中）、`after_the_middle`（后排右）。

依赖上游模块：Pipeline 模块（由 `pipeline.py` 中的 `self.classifier.classify` 调用）。

依赖下游模块：无。

分类规则详述：计算人脸框中心坐标 `(cx, cy)` 并归一化到 `[0, 1]` 区间（`cx = (bbox[0] + bbox[2]) / 2 / img_w`, `cy = (bbox[1] + bbox[3]) / 2 / img_h`），

然后按顺序遍历五个座位区域的坐标范围定义，匹配第一个命中的区域。各区域的坐标范围定义如下：

座位区域	X 范围	Y 范围	描述
posterior_left	0.25-0.45	0.25-0.45	对应后排左侧
posterior_right	0.45-0.65	0.25-0.45	对应后排右侧
after_the_middle	0.65-0.90	0.25-0.45	对应副驾驶位
co_driver	0.30-0.55	0.50-1.00	对应主驾驶位
master_driver	0.55-0.90	0.50-1.00	对应主驾驶位

当所有区域均未命中时，默认返回 master_driver。

座位区域配置数据：SEAT_ZONES 字典定义了五个座位区域的 x_range、y_range 和中文 label，作为模块级常量存储于 seat_classifier.py 文件中。

辅助方法：get_label(seat_id) 返回座位 ID 对应的中文标签；
get_color(seat_id) 返回座位 ID 对应的 BGR 颜色值，用于标注渲染。

3.3.5 标注渲染模块 (FaceAnnotator)

对应源码文件：backend/app/seat_classifier.py (FaceAnnotator 类)

模块类型：基础必选模块

模块核心职责：在原始图像上绘制人脸检测框、座位分类标签（支持中文）与置信度标注，生成可视化标注图像。使用 Pillow 库替代 OpenCV 绘图函数，以支持中文字体渲染。

输入数据：image: np.ndarray (原始图像) 与 faces: list[dict] (包含 bbox、seat、person_id、conf 等信息的人脸列表)。

输出数据：np.ndarray (标注后的图像，与输入同尺寸，OpenCV BGR 格式)。

依赖上游模块：Pipeline 模块 (由 pipeline.py 中的 self.annotator.draw 调用)。

依赖下游模块：中文字体文件 (按优先级依次尝试加载

C:/Windows/Fonts/simhei.ttf、C:/Windows/Fonts/msyh.ttf、
C:/Windows/Fonts/simsun.ttf，全部不可用时回退到 Pillow 默认字体)。

绘制逻辑详述 (draw 方法)：将 OpenCV BGR 图像转换为 PIL RGB 图像，创建 ImageDraw.Draw 对象，加载 18 号中文字体。遍历每张人脸，绘制 3 像素宽度的彩色矩形检测框 (颜色按座位区域映射)，在框上方绘制带背景色的标签文字 (格式为“座位中文名 ID:人员编号”)，在框下方绘制置信度数值。完成后将 PIL 图像转回 OpenCV BGR 格式。

座位区域叠加绘制 (draw_seat_zones 静态方法)：在图像上以半透明方式绘制五个座位区域的矩形色块与中文标签，用于展示座位分区布局。

颜色映射数据：COLOR_MAP_RGB 字典定义了五个座位区域的 RGB 颜色值
(posterior_left: (255,100,100)、posterior_right: (100,255,100)、
after_the_middle: (100,100,255)、co_driver: (255,200,100)、master_driver:
(100,200,255))，同时提供 BGR 版本 COLOR_MAP_BGR 供 OpenCV 场景使用。

3.3.6 模型权重加载模块

对应源码文件：backend/app/detector.py（_init_detector 方法内嵌逻辑）

模块类型：可选扩展模块

模块核心职责：负责加载 YOLOv8-face 深度学习模型权重文件。按优先级在多个路径下搜索模型文件：用户指定路径、当前工作目录、~/.ultralytics/weights/ 缓存目录。

输入数据：model_path: Optional[str]（用户指定的模型文件路径，为空时使用默认文件名 yolov8n-face.pt）。

输出数据：初始化完成的 YOLOv8 模型对象（self.yolo_model），或标记降级至 Haar Cascade 模式。

依赖上游模块：FaceDetector 初始化流程（在 _init_detector 中被调用）。

依赖下游模块：models/ 目录（存放 yolov8n-face.pt 等模型权重文件）。

异常处理逻辑：模型文件不存在时不抛异常，仅打印提示信息并切换至 Haar Cascade 降级方案；ultralytics 库未安装时捕获 ImportError 并降级。

3.4 文件持久化存储层模块明细

3.4.1 图片上传存储模块

对应运行时目录：data/uploads/

模块类型：基础必选模块

模块核心职责：存储用户上传的原始图片文件，作为检测处理的输入数据源。目录在服务启动时由 main.py 自动创建（UPLOAD_DIR.mkdir(parents=True, exist_ok=True)）。

输入数据：用户上传的原始图片二进制数据（JPG/PNG 格式）。

输出数据：以时间戳命名的图片文件（格式 {timestamp_ms}.jpg），供 Pipeline 后续处理使用。

依赖上游模块：FastAPI 后端调度层 main.py（API 路由在接收文件后调用 cv2.imwrite 保存至该目录）。

依赖下游模块：无（原始图片保存后由 main.py 直接传递给 Pipeline 处理，非从磁盘读取）。

存储命名规则：单张图片命名为 `{int(time*1000)}.jpg`（毫秒级时间戳）；批量图片同样采用时间戳命名。

3.4.2 检测结果输出模块

对应运行时目录： `data/results/`

模块类型：基础必选模块

模块核心职责：存储经 Pipeline 处理后生成的标注结果图片，作为检测结果的持久化归档。目录在服务启动时由 `main.py` 自动创建。

输入数据：Pipeline 输出的标注后图像（`annotated_image: np.ndarray`）。

输出数据：以特定前缀命名的标注图片文件，供后续审计或数据分析使用。

依赖上游模块：FastAPI 后端调度层 `main.py`（API 路由在 Pipeline 处理完成后调用 `cv2.imwrite` 保存标注图至该目录）。

依赖下游模块：无。

存储命名规则：单张检测结果命名为 `result_{timestamp_ms}.jpg`；批量检测结果命名为 `batch_{timestamp_ms}_{index}.jpg`（`index` 为图片序号，从 0 开始）。图像以 JPEG 格式编码，质量参数为 85%。

3.4.3 算法模型库模块

对应目录： `models/`

模块类型：可选扩展模块

模块核心职责：存放 YOLOv8-face 深度学习模型的权重文件。当模型文件存在时，系统使用 YOLOv8-face 进行高精度人脸检测；当模型文件不存在时，系统自动降级至 OpenCV Haar Cascade。

输入数据：模型权重文件（如 `yolo8n-face.pt`），由运维人员手动下载放置。

输出数据：供 `FaceDetector._init_detector` 加载使用的模型文件路径。

依赖上游模块：无（由运维手动管理）。

依赖下游模块：人脸检测模块 `detector`（`backend/app/detector.py` 在初始化时从该目录或 `~/ultralytics/weights/` 加载模型）。

3.5 模块间调用依赖关系总览

以下为系统全部核心模块的调用依赖关系链（箭头表示“调用/依赖”方向）：

前端交互模块 (frontend/index.html)

→ POST /api/detect 或 /api/detect-batch

FastAPI 服务入口 (backend/app/main.py)

→ Pipeline.process_image()

→ Pipeline.process_batch()

→ data/uploads/ (保存原始图)

→ data/results/ (保存标注图)

Pipeline (backend/app/pipeline.py)

→ FaceDetector.detect()

→ FaceRecognizer.extract_embedding()

→ FaceRecognizer.match_faces_across_images()

→ SeatClassifier.classify()

→ FaceAnnotator.draw()

FaceDetector (backend/app/detector.py)

→ models/yolov8n-face.pt (YOLOv8 模型权重)

→ cv2.data.harcascades/harcascade_frontalface_default.xml (Haar 降级)

FaceRecognizer (backend/app/recognizer.py)

→ 无外部文件依赖 (纯 OpenCV 实现)

SeatClassifier + FaceAnnotator (backend/app/seat_classifier.py)

→ Windows 系统字体文件 (simhei.ttf / msyh.ttc / simsun.ttc)

四、系统数据流需求说明

4.1 外部交互对象说明

本系统的外部交互对象仅有一个：操作人员（前端浏览器）。操作人员通过浏览器访问系统前端页面，执行图片上传操作（单张或多张），浏览器将图片以 **multipart/form-data** 格式通过 HTTP POST 请求发送至后端 API，后端处理完成后将 JSON 格式检测结果返回浏览器，前端解析并渲染标注图片、人脸列表、座位分类标签与 。操作人员不直接与后端服务、算法模块或文件系统交互，所有操作均通过前端界面发起。

4.2 三类数据存储介质说明

本系统涉及三类数据存储介质，分别对应仓库中的真实目录结构：

第一类为上传图片库，对应目录 **data/uploads/**。该目录存储用户上传的原始图片文件，由 **main.py** 在接收 HTTP 请求后将图像数据写入磁盘。文件以毫秒级时间戳命名，格式为 JPEG 或 PNG。该目录在服务启动时自动创建，属于临时性存储，用于归档原始上传素材。

第二类为检测结果库，对应目录 `data/results/`。该目录存储经 Pipeline 处理后生成的标注结果图片，由 `main.py` 在 Pipeline 处理完成后将标注图像写入磁盘。单张结果以 `result_` 前缀命名，批量结果以 `batch_` 前缀加序号命名。图像以 JPEG 格式编码（质量 85%）。该目录在服务启动时自动创建，属于持久化结果归档。

第三类为算法模型库，对应目录 `models/`。该目录用于存放 YOLOv8-face 深度学习模型的权重文件（如 `yolov8n-face.pt`），由运维人员手动下载放置。当该目录下存在有效模型文件时，FaceDetector 在初始化阶段加载该模型；当模型文件不存在时，系统自动降级至 Haar Cascade 方案，不影响正常运行。该目录为可选存储，系统不强制要求该目录存在。

4.3 两条核心业务流转路径说明

业务流一（单张图片检测流程）：操作人员在前端选择“单张检测”模式，上传一张图片。前端将图片通过 `POST /api/detect` 发送至后端。`main.py` 接收图片后，解码为 `numpy` 图像数组，保存原始图至 `data/uploads/`，然后调用 `Pipeline.process_image()` 执行五步处理——人脸检测（`detector.detect` 输出 `bbox+conf`）、特征提取（`recognizer.extract_embedding` 输出 256 维特征向量）、人脸匹配（`recognizer.match_faces_across_images` 输出 `person_id`）、座位分类（`classifier.classify` 输出 `seat_id`）、标注渲染（`annotator.draw` 输出标注图）。处理完成后，`main.py` 将标注图保存至 `data/results/`，编码为 Base64 后与人脸列表、座位标签、置信度、处理耗时等信息一并封装为 JSON 响应返回前端。前端解析响应后渲染标注图片、人脸列表卡片（含座位标签与置信度条形图）。

业务流二（批量图片清洗流程）：操作人员在前端选择“批量清洗”模式，上传多张图片。前端将所有图片通过 `POST /api/detect-batch` 发送至后端。`main.py` 接收后逐张解码为 `numpy` 图像数组，过滤无效文件，然后调用 `Pipeline.process_batch()` 执行批量处理——首先遍历所有图像逐一执行人脸检测与特征提取，将所有人脸数据汇总后调用 `match_faces_across_images` 进行全局跨图聚类匹配（这是批量模式的核心优势：通过跨图特征比对可更准确地将同一人分配统一 `person_id`），然后按 `image_id` 重新分组，为每张图的人脸执行座位分类与标注渲染。处理完成后，`main.py` 将每张图的标注图保存至 `data/results/`，编码 Base64 后与统计信息（总图片数、总人脸数、不同人员数、总耗时）一并封装为 JSON 响应返回前端。前端解析响应后渲染批量统计汇总表（含每人出现次数与主要座位）、所有标注图片卡片与人脸标注列表。

4.4 算法降级分支流转逻辑说明

本系统在人脸检测与人脸识别两个关键环节均设计了算法降级机制，确保在不同部署环境下系统均可正常运行。

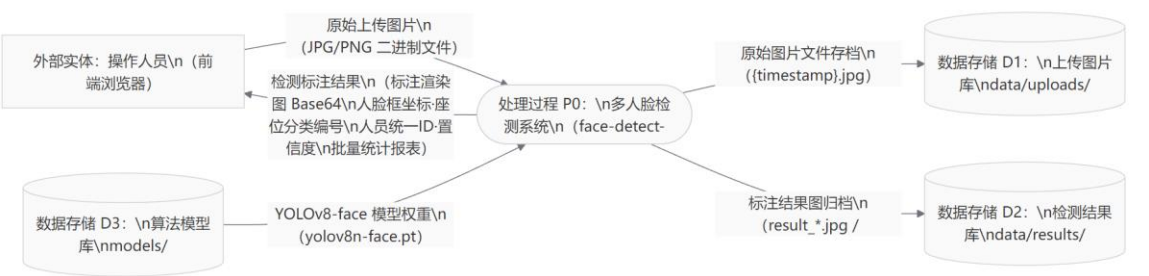
人脸检测降级逻辑：FaceDetector 在初始化阶段（`_init_detector` 方法）按优先级尝试加载检测引擎。第一优先级为 YOLOv8-face 模型：首先尝试 `from ultralytics import YOLO` 导入库，若导入成功则依次在用户指定路径、当前工作

目录、~/ultralytics/weights/ 缓存目录下搜索模型文件（默认 yolov8n-face.pt），找到则加载并设置 backend = "yolo"。当以下任一条件满足时触发降级——ultralytics 库未安装（捕获 ImportError）、模型文件在所有搜索路径下均不存在、用户显式指定了无效的模型路径——系统自动调用 _init_haar() 初始化 OpenCV Haar Cascade 级联分类器（加载 haarcascade_frontalface_default.xml），设置 backend = "haar"。Haar Cascade 为 OpenCV 内置模型，无需额外下载，始终可用。detect 方法根据 self.backend 字段动态选择 _detect_yolo 或 _detect_haar 执行检测。

人脸识别降级逻辑：项目架构设计了双方案识别体系。高级方案为 InsightFace ArcFace 深度特征提取器（输出 512 维特征向量），需安装 insightface 库并下载对应模型文件。当前实现方案（即降级方案）为纯 OpenCV 像素统计特征提取器（recognizer.py 中的 extract_embedding 方法），通过 16x16 灰度缩放+直方图均衡化+L2 归一化生成 256 维特征向量，无需下载任何外部模型文件即可运行。当部署环境安装了 insightface 并配置好模型后，可将 ArcFace 作为主方案，当前像素特征方案作为降级回退。在特征匹配阶段，当某人脸的特征向量提取失败（返回 None）时，match_faces_across_images 不会将其与已知人员匹配，而是直接分配新的递增 person_id，保证批量处理流程不因单张特征提取失败而中断。

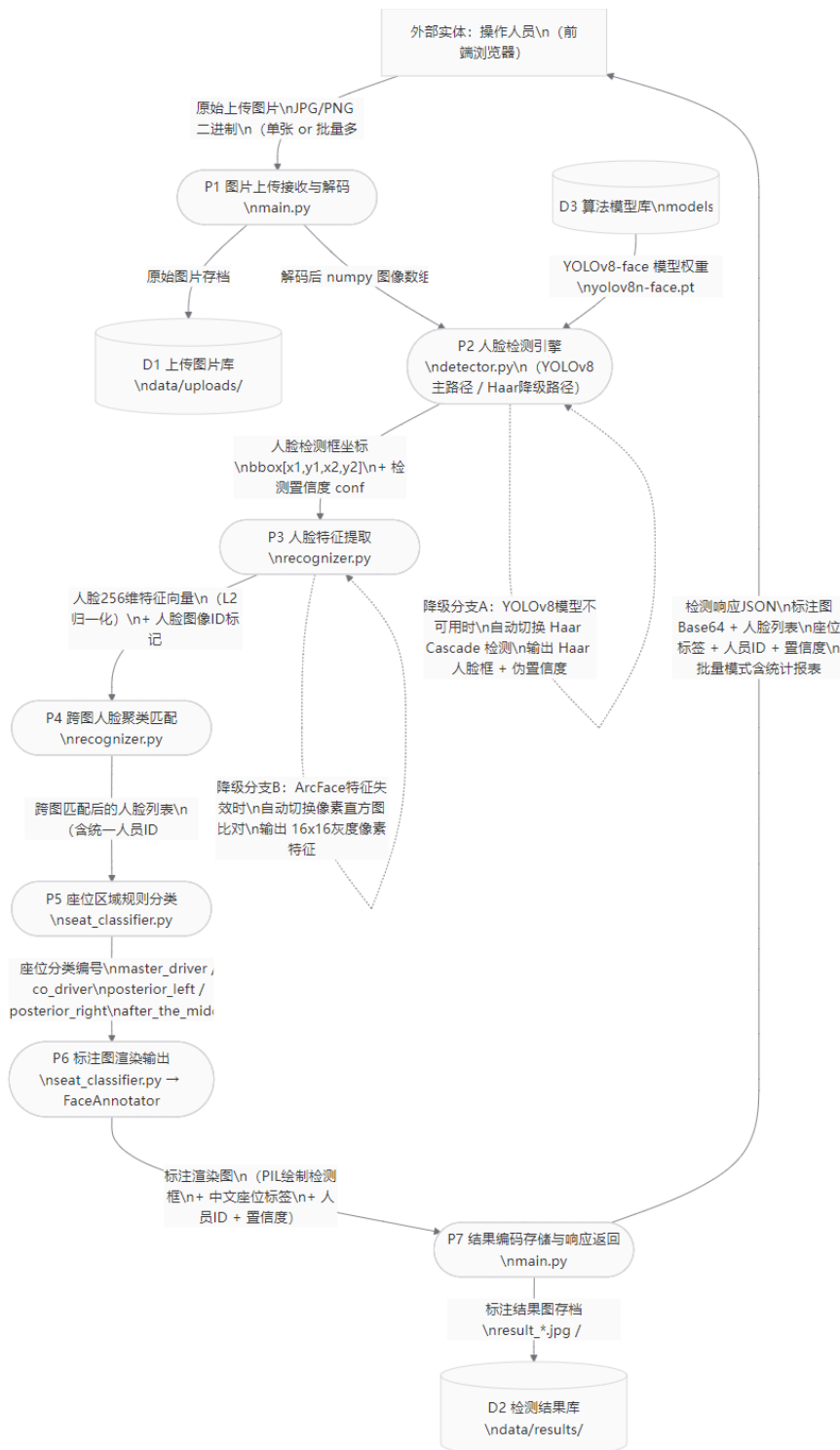
五、0 层顶层数据流图

以下为系统 0 层（顶层/上下文层）数据流图，展示系统与外部实体之间的整体数据交互关系。



六、1 层细分详细数据流图

以下为系统 1 层数据流图，展示系统内部全部业务模块之间的细粒度数据流转关系，完整覆盖两条核心业务数据流与两条算法降级分支。

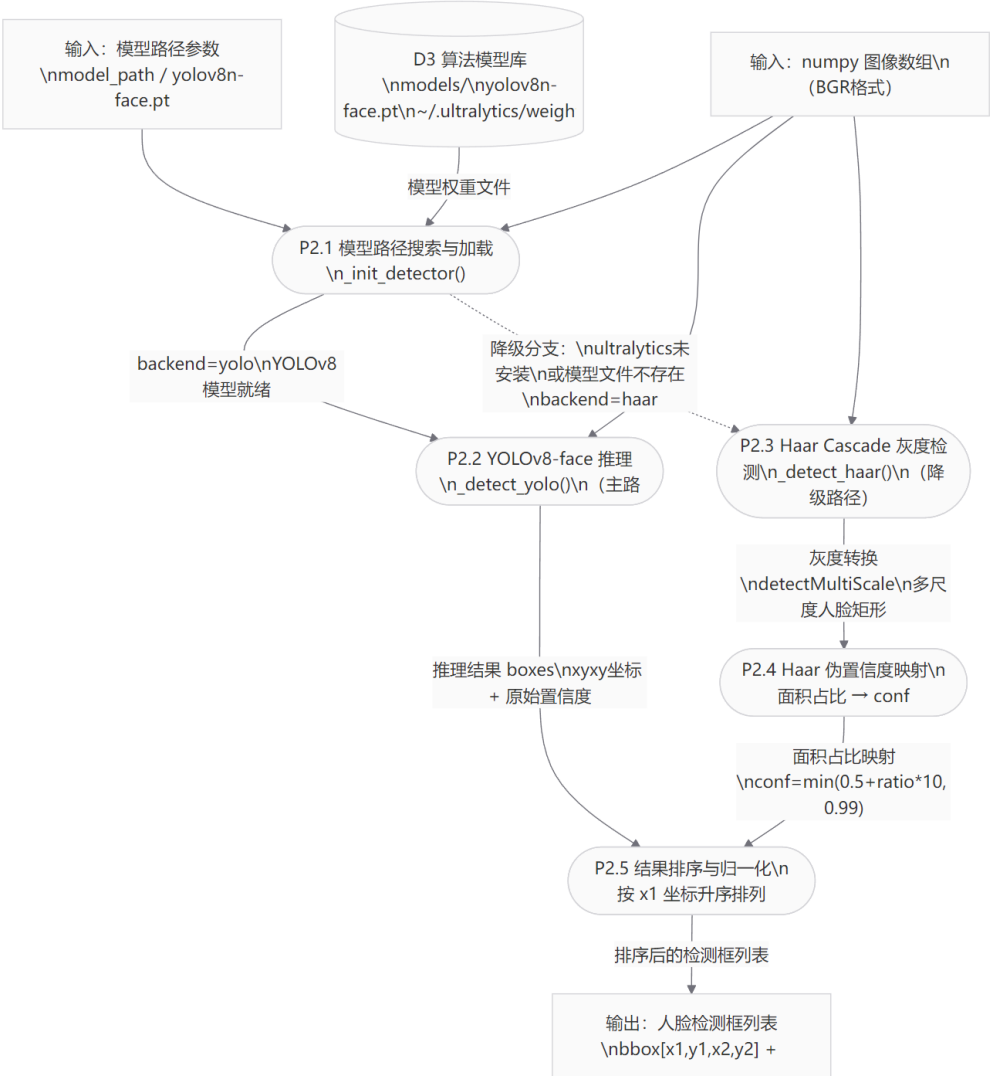


七、2 层细分详细数据流图

2 层数据流图对 1 层中的各处理过程做进一步子过程分解，展示每个核心模块内部的细粒度数据流转与处理步骤。以下按 1 层的五个关键处理过程（P2~P7）分别给出 2 层分解图。

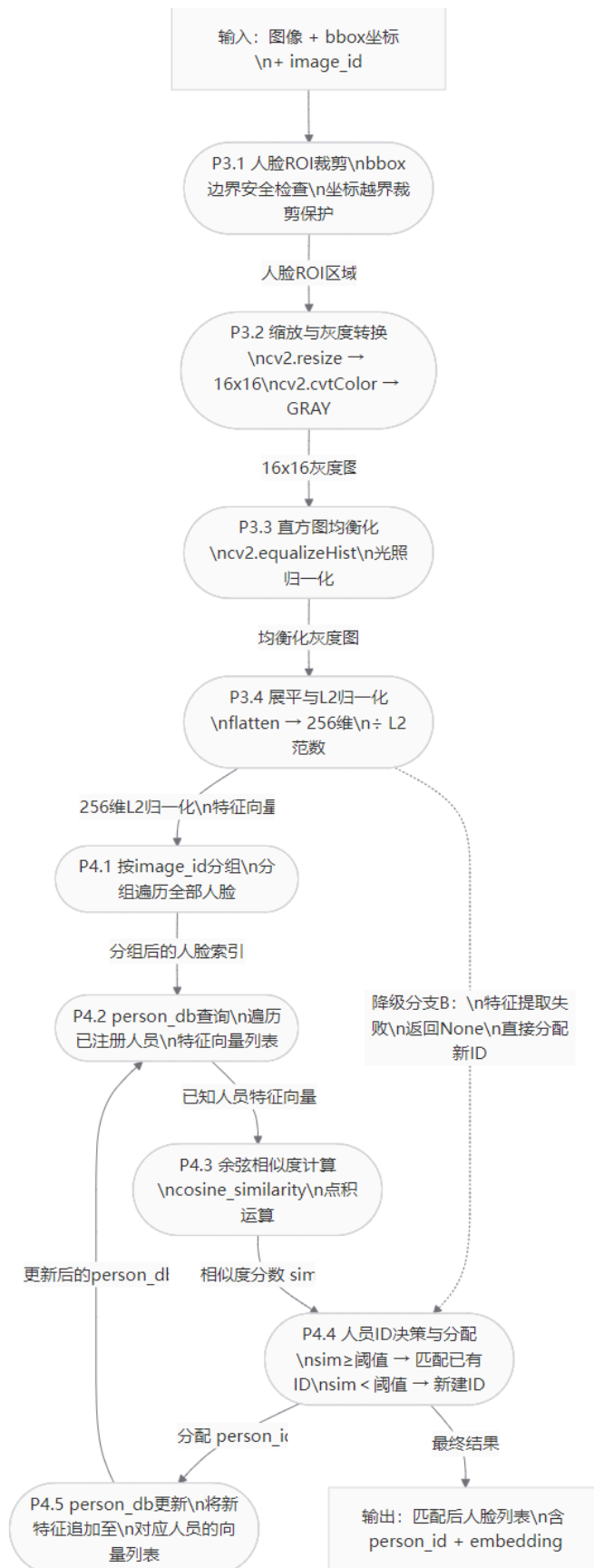
7.1 P2 人脸检测过程分解（detector.py）

本图展示人脸检测模块内部的子过程分解，涵盖模型初始化路径选择、YOLOv8 主路径推理、Haar Cascade 降级路径检测、以及置信度计算等内部数据流转。



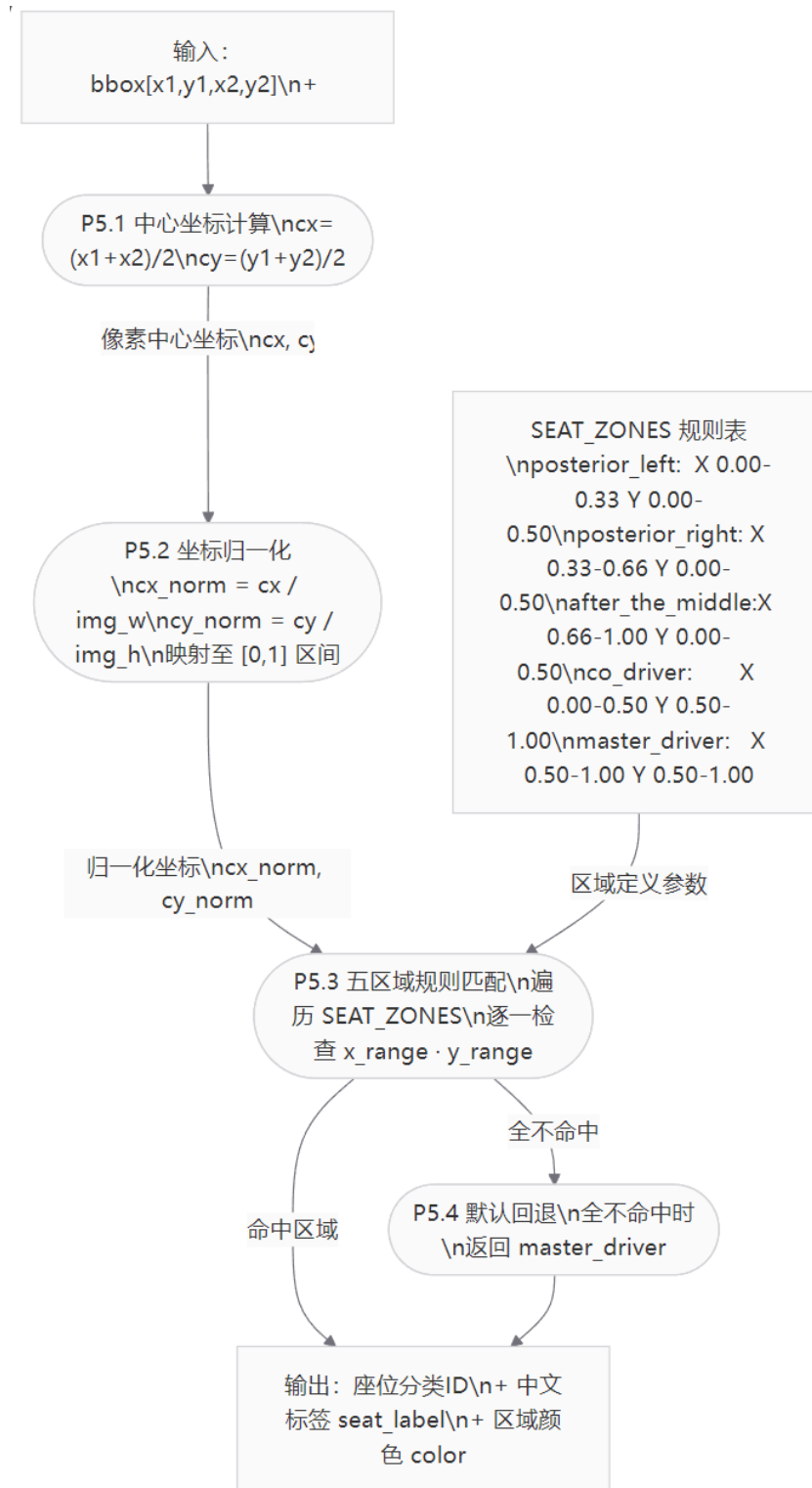
7.2 P3+P4 特征提取与跨图匹配过程分解（recognizer.py）

本图展示人脸特征提取与跨图聚类匹配的內部子过程，包括 ROI 裁剪、缩放灰度化、直方图均衡化、特征向量生成、余弦相似度计算与人员 ID 分配。



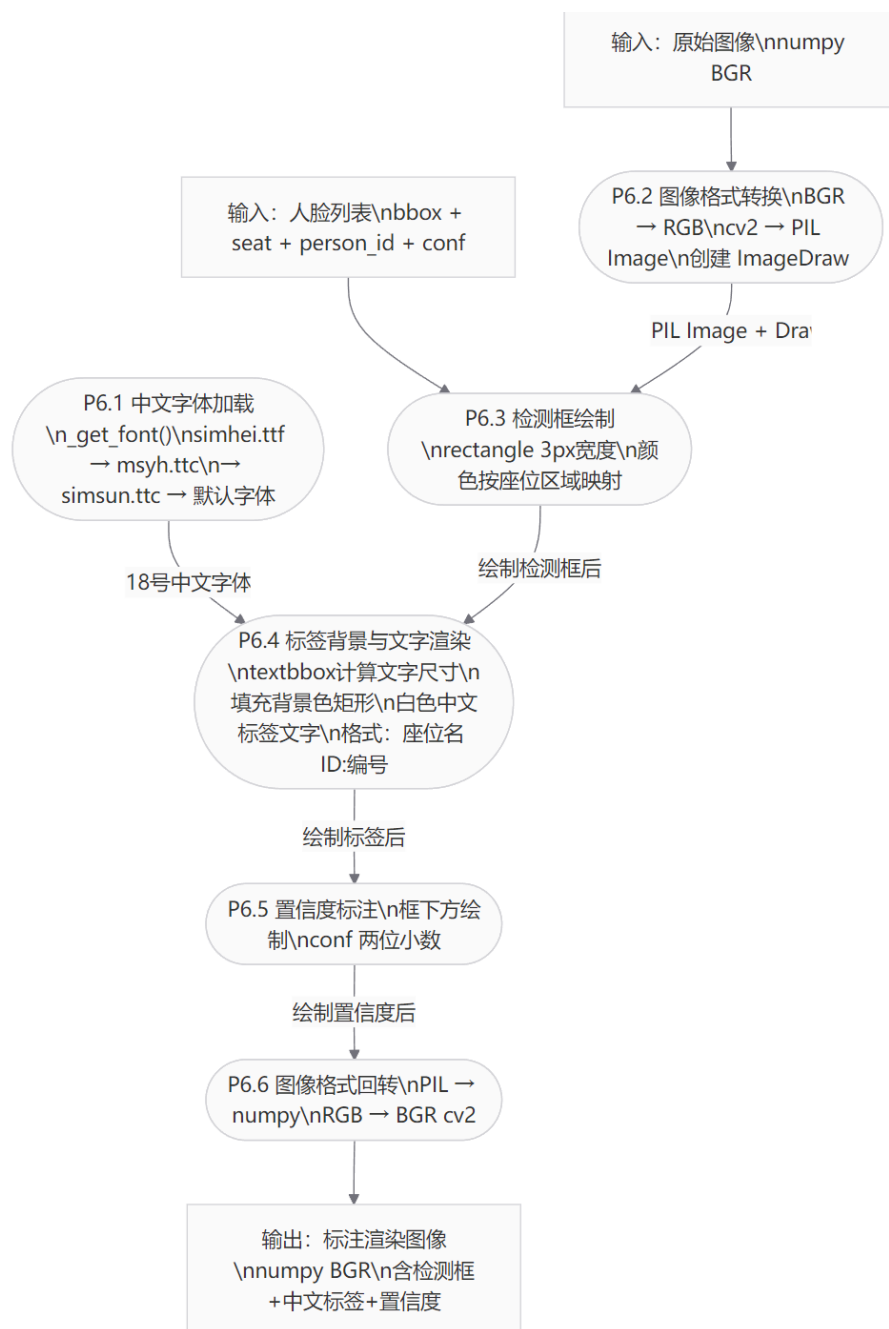
7.3 P5 座位区域分类过程分解 (seat_classifier.py)

本图展示座位分类规则引擎的内部计算步骤：中心坐标计算、归一化处理、五区域



7.4 P6 标注图渲染过程分解 (seat_classifier.py → FaceAnnotator)

本图展示标注渲染模块内部的子过程：字体加载、图像格式转换、检测框与中文标签绘制、置信度标注。



7.5 P7 结果编码存储与响应返回过程分解 (main.py)

本图展示后端调度层在 Pipeline 处理完成后的数据编码、文件存储与 HTTP 响应构建子过程

